Hybrid-INoT without compression: an evidence audit and factorial protocol for role-based code generation

Daniil Privezentsev[1*]

[1*]National Research University Higher School of Economics, Moscow, Russia.

Corresponding author(s). E-mail(s): daprivezentsev@edu.hse.ru;

Abstract

Combining several language-model roles in one call can reduce repeated context transmission, but this accounting advantage does not establish that role prompting improves software quality. We revisit Hybrid-INoT through an audit of its retained experimental records and specify a replacement study that separates these questions. Reanalysis of 240 historical Flash-Lite records confirms 49.6–90.3% lower token use for the combined, partly compressed B3 package than for a three-call B2 pipeline. However, at target contexts of 256, 1,024 and 4,096 tokens, B3 uses respectively 85.0, 37.5 and 10.3% more tokens than the simple one-call B0 comparator. Every retained success label equals one, and complete generated solutions are absent. These data establish neither a benefit of roles nor quality non-inferiority. The replacement design crosses one versus three calls with neutral versus role-labelled instructions while holding supplied context complete and identical, with compression disabled throughout. It specifies disjoint development and confirmation samples, repeated seeds, official execution-based evaluation, upstream baseline fidelity, task-level inference and complete response archives. The primary target is 1,100 held-out BigCodeBench tasks after a 40-task development split; repository repair is a separate SWE-bench study. These are prospective targets: no new confirmatory API generations or official benchmark scores are reported in this version. The contribution is a corrected empirical interpretation and an auditable, falsifiable protocol for determining whether internal role organization offers value beyond a cheaper call structure.

Keywords: Software engineering, language models, factorial design, role prompting, token budgets, reproducibility, BigCodeBench, SWE-bench

1 Introduction

A software-engineering assistant may plan a change, implement it and review the result. These operations can be placed in one model response or distributed across separate calls. They can also be described with professional role labels or ordinary procedural instructions. These choices are often changed together, making an apparently large improvement difficult to attribute. Repeated transmission of code increases input usage; additional critique may improve a patch; an output cap may prevent a final implementation from being completed. Each is a different mechanism.

The original Hybrid-INoT study compared a three-call planner–worker–critic pipeline with a combined role call, thresholded context selection and an optional rerun. Its strongest supported observation was a reduction in the resource consumption of that complete package. The study did not establish that role separation caused the saving, improved quality or transferred to real repository repair. This revision replaces the architectural superiority claim with a more specific question: *what do role instructions and call boundaries contribute when the supplied information and output allowance are controlled?*

This question matters because a complex baseline can make an intermediate system appear efficient even when a simpler solver is cheaper. Reanalysis of the retained B0 comparator demonstrates precisely that problem: at the first three target lengths, B3 incurs more tokens without an observable improvement in the saved success labels. The finding is conditional on a small ceiling-quality sample, but it changes which comparison should motivate the next experiment.

1.1 Contributions and evidence status

The present version makes three bounded contributions. First, it provides an arithmetic audit of all 240 retained Flash-Lite records, including the previously underemphasized simple baseline. Second, it derives an explicit accounting identity for an uncompressed three-stage protocol and distinguishes this identity from claims about solution quality. Third, it specifies a factorial evaluation with disjoint development tasks, observable treatment definitions, complete archives and official benchmark evaluation.

The manuscript is an **evidence audit and prospective study protocol**. Downloaded benchmark tasks, generated run manifests and software tests are preparation artifacts, not model-performance observations. No new large-scale comparison, official SWE-bench score, original-INoT replication or upstream-agent superiority result is reported here. The earlier routing rule and small-model screening policy are outside the revised contribution; they require separate interventions and are removed from the main method.

2 Related work and the remaining question

2.1 Internal roles are prior art

Sun and Zeng's INoT uses a program-like prompt to organize an internal dialogue [1]. Consequently, the existence of several role names within a single response is not a

new architectural primitive. The relevant question is whether a specified implementation offers a measurable advantage over equally structured neutral instructions and over the original method. The published INoT v1 also contains a loop-condition ambiguity: its code uses a logical disjunction where the accompanying bounded-iteration interpretation suggests a different stopping behavior. A replication must document its chosen interpretation; a silently repaired or substantially shortened prompt is an adaptation.

An emitted plan or critique is an observable output, not evidence that independent latent agents exist. The intervention studied here concerns prompt text and API boundaries. Claims about internal cognitive segregation would require additional evidence that ordinary task success and token counts cannot provide.

2.2 Budget controls and practical baselines

Tran and Kiela study single- and multi-agent reasoning under matched thinking-token budgets [2]. Their domain is multi-hop reasoning, so their findings do not determine code-generation outcomes. They do make clear why adding calls and adding computation cannot be treated as the same intervention. Our study distinguishes a matched completion allowance from actual usage, input transmission, monetary price and latency.

Agentless provides an upstream localization–repair–validation pipeline [3]; SWE-agent provides a tool-using interface for repository work [4]. These are relevant external system baselines. A handwritten three-role pipeline is neither of them. Their native retrieval and testing loops differ from fixed-context generation, so comparison requires both a controlled factorial experiment and a separate practical system evaluation. Published leaderboard values must not be inserted into a paired analysis with different models or tasks.

2.3 Benchmarks and evaluator validity

BigCodeBench contains 1,140 programming tasks requiring library use and detailed instruction following [5]. It addresses a broader code-generation setting than a handful of short HumanEval functions. Its difficulty for the selected models nevertheless has to be measured on development data. SWE-bench evaluates patches against repository tests [6, 7]. A surrogate function called `check()` does not measure issue resolution.

Official evaluation improves comparability but does not eliminate validity threats. A later audit identifies flawed-test and contamination concerns in SWE-bench Verified at frontier performance levels [8]. We therefore use Verified as a comparable repository benchmark, retain the distinction between infrastructure and model failures, and require temporally separated replication before broad claims about current engineering work. Neither a larger benchmark name nor an official evaluator alone establishes external validity.

3 Audit of the historical evidence

3.1 What is retained and what can be checked

The historical research implementation is identified by commit `010211ee5775185e8330ab6cde06e912c2adcb9e` [10]. The article repository retains the Flash-Lite E3 JSON and its earlier evidence snapshot [11]. The revision independently recomputes the JSON aggregates, checks unique task–architecture–seed keys, and reconciles total tokens with input/output counters and per-call usage records. The source SHA-256 and resulting audit table are stored with the analysis.

There are 20 unique HumanEval tasks, four target context lengths, three architectures and one seed, giving 240 records. These are not 240 independent tasks. The Pro pilot uses five tasks and is likewise exploratory. Additional context was constructed from other task material; it was not a measured set of necessary repository dependencies. Target lengths are experimental labels, not tokenizer-independent measurements of repository size.

No complete final solution or full API response is retained in the 240 Flash-Lite records. The revision can verify their arithmetic, but cannot replay the original pass/fail decisions. Regenerating an answer now would produce a new observation and would not recover the missing historical artifact. Accordingly, the table reports *retained success labels*, not newly verified correctness.

The saved usage records reveal another qualification: of 80 B3 records, 32 contain one call and 48 contain an additional call labelled `rerun`. B3 therefore averages 1.6 calls at each context length, whereas B0 and B2 contain exactly one and three calls. The audited B3 totals include these reruns. Its final success labels cannot be interpreted as single-attempt pass@1, and the historical contrast is not a pure three-versus-one comparison even before compression is considered.

3.2 The omitted simple comparison changes the interpretation

Table 1 presents mean total tokens directly recalculated from the retained file. B0 is a single solver; B2 is the three-call pipeline; B3 combines role prompting and thresholded context selection. All saved success labels are one.

Table 1 Historical Flash-Lite arithmetic audit. Each row contains the same 20 tasks, seed 42. Savings and overhead are ratios of mean token counts, not means of per-task percentages. Complete answers are unavailable.

Context	B0 tokens	B2 tokens	B3 tokens	B3 vs B2 saving (%)	B3 vs B0 change (%)
256	745.65	2,737.90	1,379.25	49.62	+84.97
1,024	1,650.80	5,486.80	2,270.50	58.62	+37.54
4,096	5,133.70	16,006.75	5,663.55	64.62	+10.32
16,384	19,159.50	58,110.00	5,632.90	90.31	−70.60

Three conclusions follow. First, the package saving against B2 is numerically reproducible. Second, B3 is more expensive than B0 at the first three target lengths; a benefit of role prompting cannot be inferred from those ceiling-quality records. Third, the apparent reversal at 16,384 tokens coincides with context selection in B3 and full transmission in B0/B2. It cannot establish that role prompting itself becomes more economical on long inputs.

The retained context metadata also distinguish target labels from actual transformations. Ratios of transformed to original context size are exactly one at targets 256 and 1,024, but range from 0.932 to 1.005 at target 4,096 and from 0.242 to 0.251 at target 16,384. Target 4,096 is therefore not a wholly uncompressed control. These are the implementation's saved size ratios, not independently retokenized prompts.

The near-flat B3 curve above the threshold is consistent with a capped selected context. A line fitted across the threshold mixes distinct operating regimes. Its slope is a descriptive summary of four design points, not an estimate of an architectural marginal cost holding all other mechanisms fixed. This revision removes that fitted slope from the headline result.

3.3 Why the old quality conclusion is unidentified

Small p-values for token differences do not establish preservation of quality. At a ceiling, there is no observed variation with which to determine whether role organization helps or harms harder tasks. Under an idealized independent-binomial planning model, zero harmful outcomes in $n = 20$ pairs gives the one-sided 95% upper bound

$$u = 1 - 0.05^{1/20} = 0.1391. \tag{1}$$

This is 13.91 percentage points, far wider than the previous two-point tolerance. The actual first-k sample is not an iid draw, so the calculation is a planning reference rather than a population guarantee. Repeating the same tasks at four context lengths does not multiply the independent sample size by four.

The earlier small-model screen retained aggregates rather than complete case-level decisions and rerun outcomes. Agreement on a surrogate SWE-bench path, including the reported 0.30 value, is not an official issue-resolution rate. The screen's causal effect on rerun costs remains unevaluated. Routing and parallel-tool conclusions also cannot repair the attribution problem in E3; they are removed from the current hypothesis family.

4 A compression-free treatment definition

4.1 Two factors with the same supplied information

The replacement study crosses call topology $t \in \{s, m\}$, for single and multiple, with role instructions $r \in \{0, 1\}$. All four cells receive exactly the same task statement and complete supplied context. The neutral and role conditions contain the same planning, implementation and review operations. They differ in whether those operations are assigned planner, implementer and reviewer labels.

Table 2 Primary factorial conditions. Compression, routing and evaluation feedback during generation are absent in every cell.

Condition	Calls	Labels	Execution
Single neutral	1	neutral	Three operations in one response
Single roles	1	professional roles	Same operations in one response
Multiple neutral	3	neutral	One operation per call; full history forwarded
Multiple roles	3	professional roles	Same separate operations; full history forwarded

The reviewer's topology-by-compression design would isolate the historical package's compression component. The author instead requires the new main experiment to avoid compression. The chosen topology-by-roles design holds compression absent and tests the role claim directly. It does not retrospectively decompose the historical package; that distinction is part of the estimand, not a semantic replacement of an unperformed experiment.

For the multiple-call conditions, every stage receives the original task/context and all prior stage outputs without truncation. The single-call conditions generate the three stages within one response. Call boundaries therefore also alter the representation of intermediate information. The estimated treatment is this specified protocol, not an abstract topology independently of all information flow. All prompt bytes and response-format rules must be frozen before confirmation.

4.2 Output allowance and truncation

The initial common allowance is 12,288 completion tokens per task and arm. Multiple-call arms allocate 4,096 tokens to each of three stages; single-call arms receive the entire allowance. This holds the maximum completion allowance constant. It does not hold total tokens, reasoning effort, actual spending or wall time constant. Unused stage allowance is not transferred in this initial design, and that restriction must be reported.

This partition could itself handicap a pipeline. A development-only calibration checks finish reasons and final-code completeness. If more than 5% of responses in any arm hit a length limit, confirmation stops until an amended allowance or allocation is fixed. A secondary experiment with a common allowance per stage can assess budget sensitivity, but it has a different estimand and cannot be pooled unlabelled with the primary experiment. Length-limited final outputs remain in their assigned denominator. No input is silently shortened to fit an API limit.

4.3 No adaptive search during the factorial generation

The generation stage produces one final candidate. Hidden tests are executed afterwards and never fed back into a later model call. The three recorded seeds represent separate single attempts; success in any of the three is not called pass@1. No router,

small checker, model fallback or adaptive retry decides which factorial arm runs. This fixed protocol makes treatment assignment independent of test outcomes and leaves practical tool-using systems to their separate comparison.

5 What the resource model does and does not prove

Let C be the tokenized shared input, P_j the stage-specific prompt overhead, and o_j the emitted output of stage j. With complete previous outputs forwarded, the total input of the three-call implementation is

$$T_m^{\text{in}} = 3C + \sum_{j=1}^{3} P_j + 2o_1 + o_2. \tag{2}$$

Including outputs gives

$$T_m = 3C + \sum_{j=1}^{3} P_j + 3o_1 + 2o_2 + o_3. \tag{3}$$

For a single combined call, $T_s = C + P_s + o_s$. Thus

$$T_m - T_s = 2C + \sum_{j} P_j - P_s + 3o_1 + 2o_2 + o_3 - o_s. \tag{4}$$

These are accounting identities for the specified serialization, with message overhead included in P. They require no compression assumption. Role names do not enter the repeated-context coefficient. Their possible value lies in changing success or output behavior, which requires measurement.

Equation (4) does not guarantee a positive total difference because output lengths can change. Nor does a token difference equal a dollar difference: cached input, model/provider rates and separately billed services matter. Record ordinary input, cache-read input, output and exposed reasoning counters as returned by the provider [9]. Reasoning tokens may already be included in completion usage and must not be added twice. Report provider USD as the primary monetary quantity and a price-normalized sensitivity calculation separately.

The cost of a successful task is estimated by total study spending divided by the number of successful tasks, not by averaging costs only among successes. It is undefined when there are no successes. API spending excludes environment setup, test execution, storage and human review. This paper does not extrapolate it into total cost of ownership.

6 Prospective experiments

6.1 Task sampling and independent units

The downloaded BigCodeBench v0.1.4 release contains 1,140 tasks. Source revision and SHA-256 values are recorded in the repository. Sort task IDs, randomly permute them using seed 20260908, allocate 40 to development and retain the remaining 1,100 for confirmation. This allocation is outcome-blind. Three generation seeds (17, 43, 101), four arms and two candidate model families give 26,400 planned generations and 52,800 planned calls. These counts describe a target design; the present version contains zero new confirmatory generations.

The development sample is used for API/evaluator controls, output allowance, provider capability and runtime feasibility. It is excluded from confirmatory estimates. Freeze any amendment before examining held-out outcomes. The published Hard subset can be a prespecified stratum inside the task set, but it is not a new independent dataset. Task IDs, not rows or stochastic repeats, are the unit of generalization.

The initial economical model candidates are Qwen3 Coder 30B A3B Instruct and Gemini 2.5 Flash-Lite. Their API identifiers and advertised rates were checked on 8 September 2026. Provider aliases are not immutable weights. Before a paid canary, pin a provider that supports all requested parameters, disable fallback routing, and archive the returned provider/model identity. An accepted seed is recorded without promising deterministic reruns.

6.2 Repository repair is a separate study

The secondary target is all 500 SWE-bench Verified instances, with three seeds and four factorial arms for an initial model: 6,000 planned generations and 12,000 calls. Before inference, repository files must be collected at the instance's `base_commit` using an outcome-blind retrieval rule. The same complete retrieval packet is used across arms. Retrieval selects files, so "uncompressed" means the supplied packet is preserved; it does not imply that an entire arbitrary repository fits in every model window.

Gold patches, test patches, future repository commits and evaluator-only metadata cannot enter the prompt. The current generation interface does not itself implement this repository retrieval. End-to-end claims require the retrieval stage, generated patch, application logs and official tests to be present together. This secondary study is not represented by a function-generation smoke test.

6.3 Official evaluators and controls

BigCodeBench candidates are exported in the official sample format and evaluated in a pinned isolated environment. For SWE-bench, each prediction contains the instance ID, model identifier and patch. The official harness applies the patch and produces case-level reports [7]. Before scoring model outputs, execute a reference/gold control and a deliberately failing control, and inspect the resulting test reports. An evaluator process that exits successfully is not itself evidence that a candidate passed.

Persist test output, result JSON, commands, source version, environment/image digest and the exact evaluated code/patch hash. Invalid or empty generated code is

a model failure. Environment construction failures are infrastructure outcomes and remain identifiable. A fresh evaluation identifier derived from prediction bytes prevents accidental reuse of a cached result for a changed patch. The present version does not report that these official evaluation controls or model evaluations have completed.

6.4 Baseline fidelity

The full comparison requires a direct one-call solver, original INoT, Agentless and SWE-agent in addition to the four causal cells. Direct solving tests whether structured operations offer any benefit at all. Original INoT tests whether the revision adds value beyond its intellectual source. The two upstream repository systems test practical relevance with real retrieval and tools.

For each baseline, record source commit, exact prompt/configuration, model, provider, task IDs, budget overrides, returned trajectories and official evaluation artifacts. Preserve native behavior when possible. If a native pipeline uses candidate search or intermediate tests, count those calls and disclose that this is a system-level comparison. A renamed in-house pipeline, an INoT-inspired paraphrase or a published percentage cannot satisfy the baseline requirement. No new baseline outcome is claimed in this revision.

6.5 Feasibility and the fixed spending envelope

The author-set OpenRouter envelope is USD 300 across all phases, retries and resumed runs. Initial allocations are USD 10 for development, USD 190 for the primary series, USD 75 for repository/baseline pilots and USD 25 reserved for uncertainty. These are ceilings, not proof that the desired matrices fit. Conservative request envelopes and actual canary usage must be checked before freezing a feasible confirmatory allocation.

If resources support only a subset, it must be selected outcome-blind and explicitly reported as a smaller exploratory series. Running until funds expire and publishing the observed prefix as the planned large study would reintroduce selection bias. The initial key-status check found no remaining allowance; no new paid generations were performed at that stage. Unknown request charges reserve their entire permitted envelope until reconciled.

7 Statistical analysis plan

7.1 Estimands and hypotheses

Let $Y_{i,t,r,k}$ be the binary official-test outcome for task i, topology t, role condition r and seed k. Average over the fixed seeds to obtain $\bar{Y}_{i,t,r}$. Define the role effect within topology and the topology effect within a role condition as

$$\Delta_R(t) = \mathbb{E}_i[\bar{Y}_{i,t,1} - \bar{Y}_{i,t,0}], \tag{5}$$

$$\Delta_T(r) = \mathbb{E}_i[\bar{Y}_{i,s,r} - \bar{Y}_{i,m,r}], \tag{6}$$

$$\Gamma = \Delta_R(s) - \Delta_R(m). \tag{7}$$

The interaction Γ asks whether the effect of role instructions depends on the call protocol. A main effect alone cannot answer that question. Report all four cell means and these five contrasts for each model and benchmark. Analogous absolute-dollar contrasts describe cost; log-cost contrasts require strictly positive observed costs and are secondary.

The role-benefit hypothesis concerns $\Delta_R(s)$ and is evaluated with a two-sided test and interval: a negative or null result is admissible. The efficiency hypothesis compares single-role and multiple-role USD costs. Quality preservation is a separate non-inferiority claim,

$$H_0 : \Delta_T(1) \leq -0.02, \qquad H_1 : \Delta_T(1) > -0.02. \tag{8}$$

Declare preservation only when the one-sided 97.5% lower bound exceeds -0.02. The advertised cost–quality tradeoff requires both an efficiency finding and that quality criterion. Nonsignificant superiority testing is not evidence of non-inferiority.

7.2 Dependence, multiplicity and missing observations

Resample tasks, retaining all their arms, seeds and paired models together, with 10,000 bootstrap draws and seed 20260908. Seeds are repeated measurements, not independent tasks. Analyze models separately; any pooled model estimate must specify its weighting. For the five quality contrasts, apply Holm correction within each prespecified benchmark/model family. Report estimates and uncertainty even when corrected tests are nonsignificant.

For sparse discordance, report the paired outcome table for the prespecified seed 17 and conservative exact bounds in addition to bootstrap summaries. The conservative release gate requires both the bootstrap lower bound and the first-seed bound to exceed -0.02. The latter subtracts an upper Clopper–Pearson bound for harmful outcomes from a lower bound for beneficial outcomes, allocating probability 0.0125 to each tail. It is not an exact interval for the multi-seed mean. A zero-width bootstrap interval on an all-success sample does not establish quality preservation. Paired sign-flip tests are reported as a symmetry-based sensitivity analysis. The analysis can return "inconclusive" even after all planned cells are completed.

Empty outputs, invalid code and unsuccessful patches count as model failures. Infrastructure outcomes are recorded as missing, with separate optimistic and pessimistic bounds in which unresolved observations are assigned success or failure. Primary complete-task contrasts require every expected seed/arm cell and must disclose the excluded count. A complete-case result cannot silently replace the planned intention-to-evaluate denominator. No result-based task deletion, prompt tuning or selective seed reporting is allowed.

7.3 Power: why a large grid may still be insufficient

For a single seed, let the paired outcome difference be $D \in \{-1, 0, 1\}$ and let d be the probability of discordance. Near zero mean, $\mathrm{Var}(D) \approx d$. A normal-approximation planning requirement for 80% power at the two-point margin and one-sided $\alpha = 0.025$

is

$$n \approx \frac{(z_{0.975} + z_{0.80})^2 d}{0.02^2}. \tag{9}$$

Table 3 Approximate independent-task requirements from Eq. (9). These are planning calculations, not achieved power.

Discordance d	0.05	0.10	0.20
Required tasks, rounded up	982	1,963	3,925

The previous 149-pair calculation concerned a best-case one-sided 95% zero-harm binomial bound; it was not an 80%-powered non-inferiority design. Even 1,100 tasks do not guarantee sufficient precision. Use development-only nuisance estimates to simulate the repeated-seed design before freezing it. If the available budget cannot support the required precision, report effect estimates and uncertainty without weakening the margin after observing results.

8 Reproducibility as an evidence chain

Reproducibility requires more than a repository commit and a hash of aggregate numbers. The evidence chain links frozen input, exact API request, returned response, extracted final candidate, executed tests and the analysis row. A hash is useful only if the hashed artifact is actually retained and accessible.

Before every call, persist the request without authorization headers and reserve its maximum permitted cost. Immediately retain the complete response body, provider identifiers, usage and finish reason before parsing. Exposed response text is stored in full; the protocol does not claim access to undisclosed model reasoning. Persist intermediate outputs before the next call so that a later failure cannot erase earlier evidence. Resume only when task, configuration, prompt/source and manifest hashes match, with no unaccounted pending charge.

Archive audit checks expected task–model–arm–seed coverage, duplicate keys, byte integrity, token/cost reconciliation and candidate-to-test linkage. The historical files are preserved as a distinct legacy dataset; no regenerated answer is inserted into them. Invalid extraction is an observable failure, not a license to choose a more successful block manually.

The implementation and versioned protocol accompany this manuscript in the article repository [11]. Unit tests and preparation checks test the software's invariants. They must not be included in the experimental sample or interpreted as evidence that a model solves benchmark tasks. Source code for missing repository retrieval, completed upstream-agent runs and complete official evaluator records remain required before publication of an empirical superiority claim.

9 Interpretation, limitations and publication boundary

The corrected historical conclusion is narrower and more useful than a generic efficiency claim. Combining calls and selecting context was cheaper than the historical three-call package. At the first three target lengths, the simple solver was cheaper still. Which organization offers the best cost–quality tradeoff is therefore unresolved, not automatically favorable to internal roles.

The new factorial study can identify effects of its frozen prompts and call protocol within its task population. It cannot establish latent independence of roles, universal benefit across providers, correctness beyond the tests or performance on an unseen repository distribution. Matching an output allowance leaves input/caching and actual computation unequal. Fixed stage budgets can interact with difficulty; fixed context packets exclude adaptive information gathering. Both limitations motivate separate sensitivity and practical-system comparisons.

The benchmark sample is large relative to the historical pilot, but its target size is prospective. Data contamination, evaluator defects, dependence between related tasks and provider drift remain relevant even after full execution. Repository repair additionally requires real retrieval and a functioning isolated harness. The complete absence of new confirmatory outcomes in this version prevents a claim that the major empirical review concerns have already been closed.

10 Conclusion

The retained evidence supports a package-level reduction in tokens against a three-call baseline, while a recovered comparison with the simple solver weakens the case for role prompting itself. We replace the confounded claim with a compression-free factorial question, explicit cost identities and an evaluation protocol that can reject the proposed benefit. A publishable empirical conclusion must come from complete model responses, official tests, credible baselines and task-level uncertainty. This revision supplies the corrected interpretation and study specification; the large-scale confirmatory result remains to be obtained.

Data and code availability

The article repository [11] contains both language versions, PDFs, historical records, the arithmetic audit, source/dataset hashes and the prospective protocol. Historical research code is pinned separately [10]. Official benchmark data and upstream evaluators retain their respective licenses. Model-response and test artifacts for the new study are not available because the confirmatory runs have not been completed.

Declarations

Funding and competing interests: no external funding or competing interest was reported in the preceding manuscript. **Research scope:** public software benchmarks; no human-participant study is reported. **Author responsibility:** the author

is responsible for final scientific claims and for approving the completed experimental record. AI assistance was used in manuscript revision and research-software preparation; software checks are distinguished from experimental results.

References

[1] H. Sun and S. Zeng. Introspection of Thought Helps AI Agents. arXiv:2507.08664v1, 2025. https://arxiv.org/abs/2507.08664v1.

[2] D. Tran and D. Kiela. Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. arXiv:2604.02460v2, 2026. https://arxiv.org/abs/2604.02460v2.

[3] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024. https://arxiv.org/abs/2407.01489.

[4] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS, 2024. https://arxiv.org/abs/2405.15793.

[5] T. Y. Zhuo et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. ICLR, 2025. https://arxiv.org/abs/2406.15877.

[6] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. https://arxiv.org/abs/2310.06770.

[7] SWE-bench maintainers. Evaluation Guide. Accessed 8 September 2026. https://www.swebench.com/SWE-bench/guides/evaluation/.

[8] OpenAI. Why SWE-bench Verified no longer measures frontier coding capabilities. 2026. https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/.

[9] OpenRouter. Models API and provider usage metadata. Snapshot accessed 8 September 2026. https://openrouter.ai/api/v1/models.

[10] D. Privezentsev. INoT_Research. Historical implementation, commit `010211ee5775185e8330ab6cde06e912c2adcb9e`. https://github.com/daniil-novel/INoT_Research.

[11] D. Privezentsev. Article-INoT: manuscript, historical artifacts and compression-free revision protocol. 2026. https://github.com/daniil-novel/article-INoT.